Define functions and return values

Recently, we've been exploring variables, expressions, and data types. In this video, we'll consider another important component of programming in Python: functions. A function is a body of reusable code for performing specific processes or tasks. We've come across a few built-in Python functions so far. For instance, the print function writes text on the screen, the type function tells us the data type contained within a variable, and the S-T-R function converts an object into a string. Note that in previous version of Python, print was handled as a statement and did not use parentheses. But, for Python 3, the print syntax is a function and requires parentheses, even when there are no arguments used and the parentheses are empty.

Okay, so we know that Python has many built-in functions, but if we want to tell a computer to do other things particular to our own use cases, it's important to know how to define our own functions. To define a function, we use the keyword word def at the start of the function block. You've encountered the define function once before in a previous video, but let's consider another example. When defining a new function, always begin with the def keyword. The name of the function comes next. Let's call it "greeting." After that, we have the function arguments, also known as parameters.

A function's arguments are always written in parentheses. The arguments are the things you give to the function to modify in some way. You can call them anything you want. Whatever we call them here when we define the function is how we'll have to refer to them below in the function's body. In this example, our function will have just one parameter: "name." When we're done defining the arguments, close the parentheses, put a colon at the end, and hit Enter to get to a new line. Now we can write the body of the function.

This is where we say what we want the function to actually do. Note how the body is automatically indented to the right. In Python, lines of code or hierarchical. Any line that is indented pertains specifically to the less-indented code that precedes it. We can add as many lines as we'd like to the body of the function, but each line must be indented to the right. Here, it's indented four spaces. You can use however many spaces you like, as long as you're consistent.

However, four spaces is usually the preferred way because it makes code more readable. Our "greeting" function will take a name and output a greeting using that name. We'll have the function print "Welcome," the person's name, and then on a new line, print "You are part of the team." To finish defining the function, simply unindent the next line of code. Now we can call the function using the word "greeting." Inside the parentheses, we'll type the name "Rebecca." Then we'll run the cell.

Of course, functions can do a lot more than print messages. This is just one simple example of defining your own function. Next, let's consider how to get values out of a function. This is where return values can be used. Return is a reserved keyword in Python that makes a function do work to produce new results. But instead of printing the

results, the function saves them for later use. Let's define a new function that accepts two arguments, the base and height of a triangle, and returns the area of the triangle. The area is calculated as base times height divided by two. We use the keyword "return" to tell Python that this is the value that we want to come out of the function. Instead of printing, return lets us store this value in a variable. So, suppose we have two triangles and want to add the sum of both areas. Here's what we would do: First, calculate the two areas separately, storing each value in its own named variable. Then add the two areas together, assigning the results to a variable called "total_area." If we call this variable, the Jupyter Notebook returns its value, but we don't have to call it. We could continue writing code if we want. This demonstrates the power of the return statement. It enables us to combine function calls with other operations, which makes the code reusable. Reusability involves defining code once and using it many times without having to rewrite it. There's more information on reusability soon, but for now, just understand that reusing something takes a lot less time and effort than recreating it every time. Let's do one more. Here's a function called "get_seconds."

This function takes hours, minutes, and seconds as inputs and returns the total number of seconds those inputs represent. In the first line, we begin with the keyword word "def" and name the function "get_seconds." In the parentheses, we give it three parameters: hours, minutes, and seconds. The next line performs a computation that calculates the total number of seconds and assigns that value to a variable called "total_seconds." The third and final line is the return statement that returns the value of "total_seconds."

When we call the function, we have to give it three arguments: hours, minutes, and seconds. We'll use 16 hours, 45 minutes, and 20 seconds.

And there's our result, 60,320 seconds. Now you understand more about functions and how to use the return keyword to save the results of a function for later output. Code reuse is a key element of Python that you will continue to appreciate as a data analytics professional. Your data toolbox is growing and growing, and there's more on the way.

Function

A body of reusable code for performing specific processes or tasks

def

A keyword that defines a function at the start of the function block

return

A reserved keyword in Python that makes a function produce new results, which are saved for later use

Reusability

Defining code once and using it many times without having to rewrite it

Question

Fill in the blank: A _____ is a body of reusable code for performing specific processes or tasks.

- variable
- function
- data type
- keyword

Correct
A function is a body of reusable code for performing specific processes or tasks.

Continue Skip